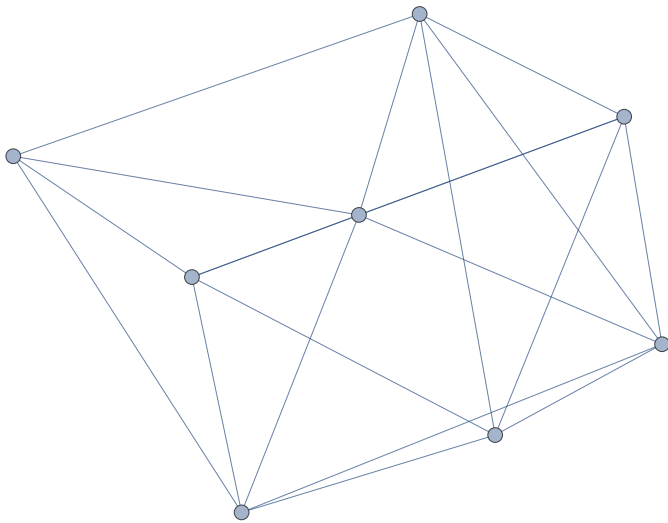


This is a random graph with 8 vertices and 20 edges.

In[2]:= **RandomGraph**[{8, 20}]

Out[2]=



Graphs

A graph is given by a set of vertices and a set of edges.

An edge is a pair of vertices.

Finite Graph

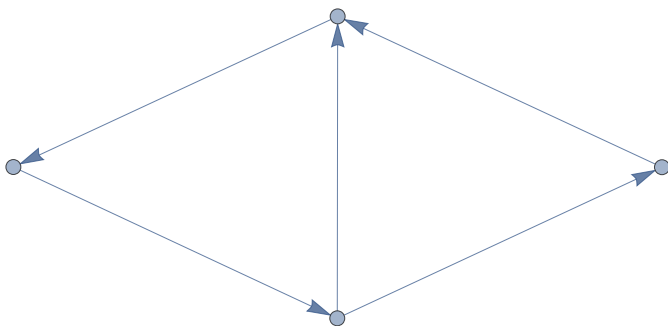
A finite graph has a finite set of vertices and a finite set of edges.

Directed Graph

If the edges are ordered, then it is a directed graph. Usually such a graph is drawn with arrows for its edges.

In[4]:= **Graph**[{1 → 2, 2 → 3, 1 → 3, 3 → 4, 4 → 1}]

Out[4]=

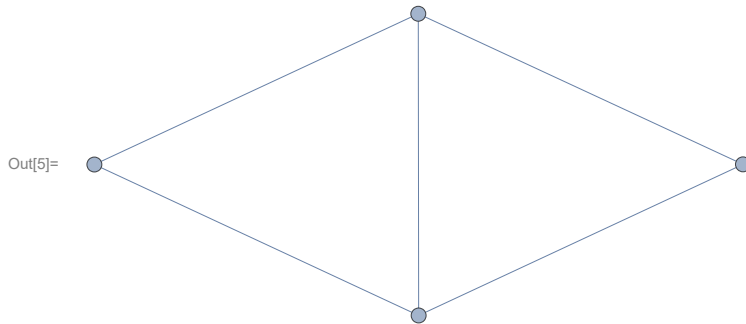


Undirected Graph

If a graph has unordered edges then it is an undirected graph.

Usually the edges in an undirected graph are drawn with line segments.

In[5]:= **Graph**[{1 ↔ 2, 2 ↔ 3, 1 ↔ 3, 3 ↔ 4, 4 ↔ 1}]



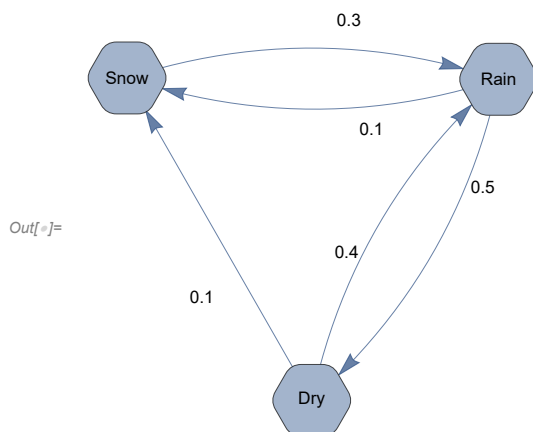
Types of applications

Whenever there is a notion of connectivity, then a graph is appropriate. Atoms in a molecule, neurons in a nervous system, computers on a network, etc..

Graphs can also be used in more abstract situations to describe how items are related. The changing states of a computer program, set inclusion, political hierarchies, data structures, etc..

Graphs can also show flows or how things change. Often the edges come with other properties, like how much flows from one vertex to another.

Here for instance are the chances of a snowy day being followed by a rainy day, etc. in a hypothetical situation. Each edge has a number interpreted as a probability or proportion, depending on whether the application is probability or statistics.

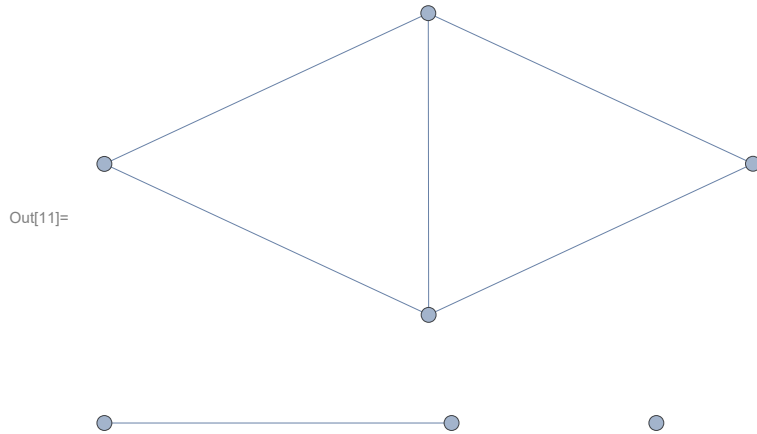


Weighted Graph

The edges of a weighted graph have a numerical value for a property.

Disconnected graphs

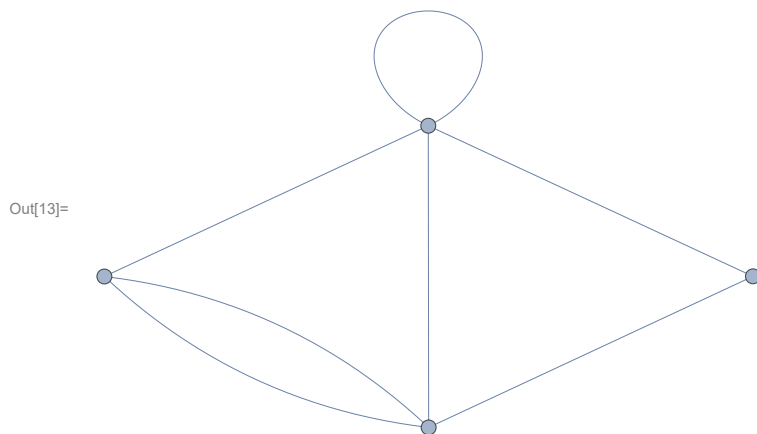
A graph need not be connected. There may be a vertex with no edges.



Loops and multiple edges

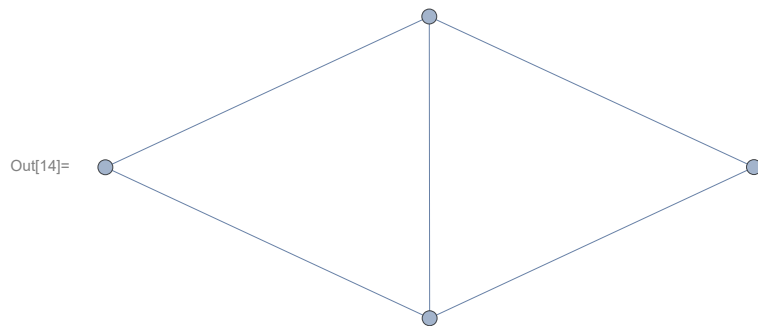
In general, it is possible for a graph to have loops or multiple edges.

In[13]:= **Graph**[{1 ↔ 2, 2 ↔ 3, 1 ↔ 3, 3 ↔ 4, 4 ↔ 1, 1 ↔ 1, 2 ↔ 3}]



Simple Graph

A simple graph is an undirected graph with no loops and no multiple edges. Some authors require a simple graph's vertices and edges to have no properties.



A simple graph may be disconnected.

Degree

The degree of a vertex is the number of edges connected to it.

In a directed graph there can be a distinction between out degree and in degree.

Some authors will call this vertex degree.

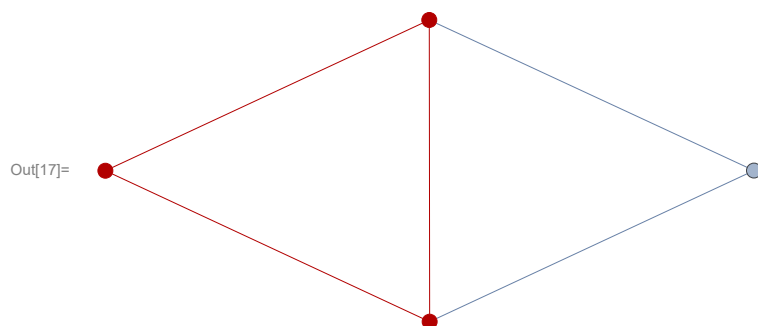
Path

A sequence of vertices such that $x_n \leftrightarrow x_{n+1}$ is an edge

If the edges are directed then $x_n \rightarrow x_{n+1}$ has to go the same way as an edge in the graph.

Cycle

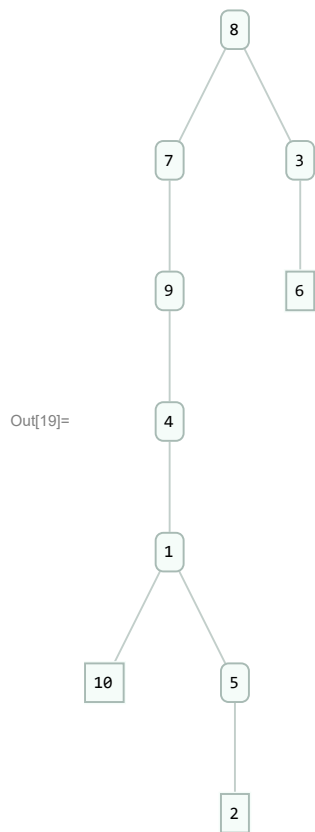
A cycle in a graph is a path which ends where it starts.



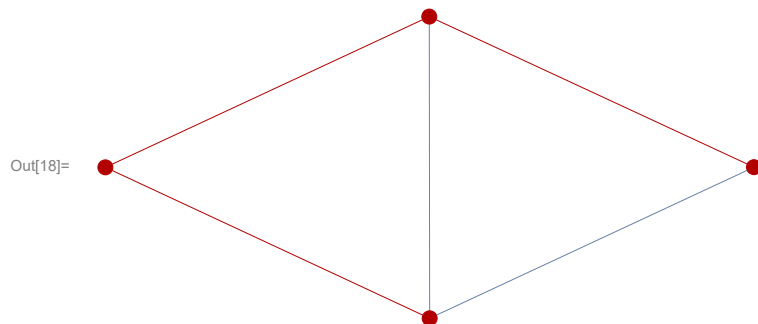
Tree

A tree is a graph with no cycles. A data structure can usually be represented by a tree.

In[19]:= **RandomTree[10]**

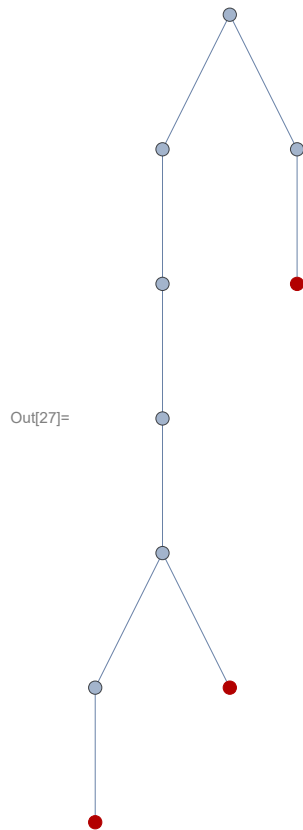


Every graph has a *spanning tree* (usually more than one).



Leaves

The leaves of a tree are the vertices with only one edge.



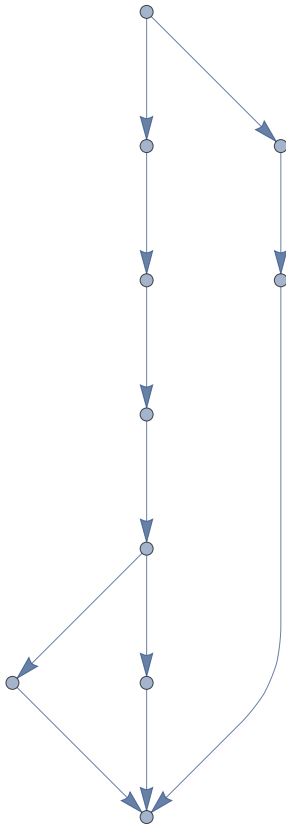
Root

One vertex of a tree can be specified as a root if the tree is connected. Usually the choice of a root determines the way the tree is drawn.

Directed Acyclic Graph (DAG)

A directed acyclic graph is a directed graph with no cycles.

Out[28]=



Graph distance

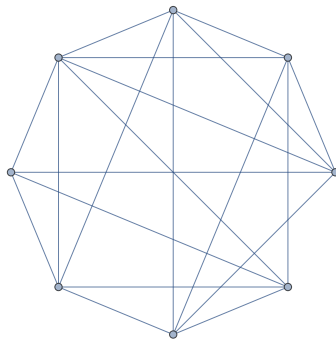
The distance between two vertices x and y is the smallest number of edges of any path from x to y .

Graph layouts

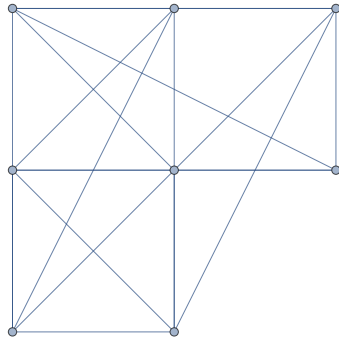
There are many ways to display the same graph.

Usually one wants a clear picture, but it could also have some sort of symmetry for the vertices.

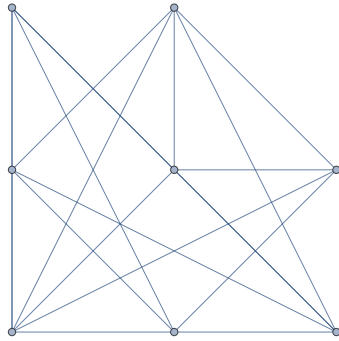
CircularEmbedding



DiscreteSpiralEmbedding

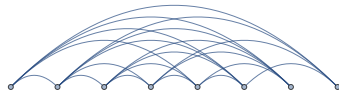


GridEmbedding

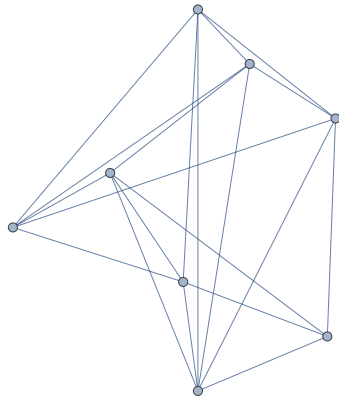


Out[38]=

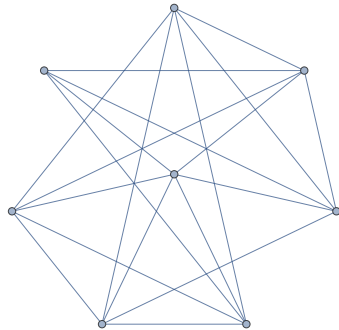
LinearEmbedding



SpiralEmbedding

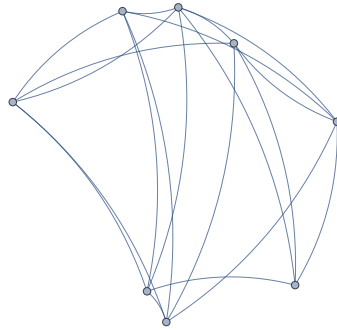


StarEmbedding

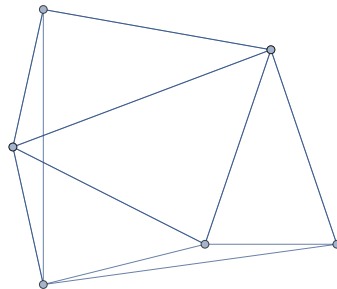


Some embeddings attempt to minimize a function, like the spring embedding imagines that there are springs for edges and the result would be in equilibrium.

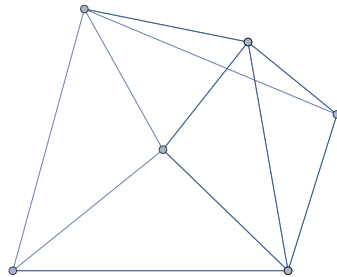
GravityEmbedding



HighDimensionalEmbedding

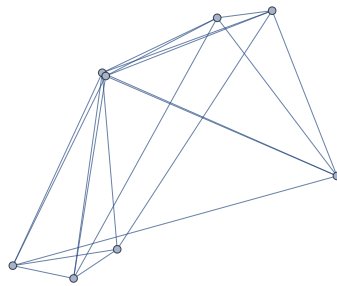


SpectralEmbedding

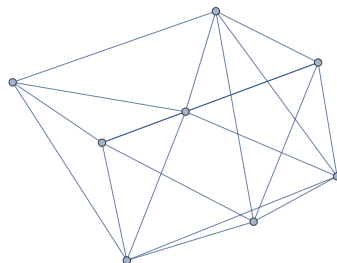


Out[43]=

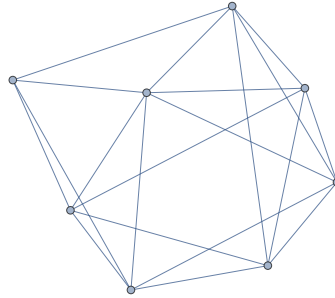
SphericalEmbedding



SpringElectricalEmbedding



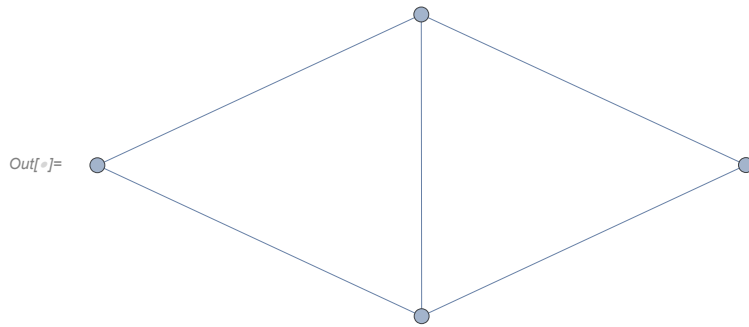
SpringEmbedding



Graph Representations

A list of unordered pairs can represent a graph.

```
In[ ]:= Graph[ {1 ↔ 2, 2 ↔ 3, 1 ↔ 3, 3 ↔ 4, 4 ↔ 1} ]
```



Adjacency Matrix

A matrix of 0's and 1's is called an *adjacency matrix*.

```
Out[44]//MatrixForm=
```

$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

The entry a_{ij} is in row i and column j

$$a_{ij} = \begin{cases} 1 & \text{if there is an edge from vertex } i \text{ to vertex } j \\ 0 & \text{if there is no edge from vertex } i \text{ to vertex } j \end{cases}$$

The adjacency matrix of a simple graph is symmetric: $a_{ij} = a_{ji}$

One convenient means of representing a graph is by its adjacency matrix.

If there are a large number of vertices, the adjacency matrix can be big. There could be a lot of 0's in the adjacency matrix, for instance if the vertex degree is small.

Sparse Matrices

Sparse matrices are usually implemented as a list of unordered pairs, but interpreted by the computer

as a matrix. This can reduce the storage requirements, and increase the speed of matrix operations.

Depending on the implementation, the edges could be represented by rules based on the vertex number, or some other form of compression.

```
In[45]:= AdjacencyMatrix[RandomGraph[{80, 120}]]
```

```
Out[45]= SparseArray[
```



Specified elements: 240

Dimensions: {80, 80}

Default: 0

Density: 0.0375

Elements:

{1, 3} → 1

{1, 34} → 1

{1, 65} → 1

{1, 69} → 1

⋮